Are you going to share slides or something? Yeah, yeah, I'm gonna share my screen. Um, just a minute. Okay, it's, uh, I've never shared all Microsoft teams before, so it's telling me I need to quit and reopen the app, so I'll, like, come back in, like, a minute. Okay, okay. Okay, uh, try sharing again. Uh, can you guys see? Um, I think you ... Sorry? Oh, you meet yourself? Oh, I am. And I'll mute the audio. Sorry, can you guys, can you guys see my screen? Yeah. Okay, cool. So, um ... So, The main thing I tried to do over the last 2 weeks was kind of avoid jumping directly into optimization before understanding like runtime or model structure like properly. So I approached it in like the following stages. Um, I tried to 1st understand the open vino runtime stack. Then I wanted to, um, trace the uniform VLA Ferris architecture, validate the export blocker assumptions. The, um, which I mentioned in my proposal, and then finally, like, start prototyping the actual, like, runtime architecture. Um, Or like that I thought like kind of made sense for deployment. And, um, the main result so far was that I was able to, um, isolate the repeated DIT denosing step. Um, export it into uh, the open veno intermediate representation. Uh, then validate runtime execution and then start rebuilding uh, the de-noisy scheduler around the uh, compiled step. So overall, like work has been moving about from like understanding to like validating, at least for the past, um, a week or so. So, here's what I kind of like learned. Uh, so I started by studying, um, the open vino, like runtime architecture, because I wanted to avoid treating open vino as just kind of like a random and friends library. I just wanted to understand like how exactly it worked. So, my understanding now is open vino, it acts as a compiler stack, uh, like a pie torch graph gets converted into an uh, open vino IR. Uh, and then the graph goes through optimization as uh, and and different like fusion passes, then it gets compiled for a hardware plug-in, whether it's like CPU, GPU, or MPU. And finally, um, execution gets dispatched through, um, like, like primitives and kernels. Um, I also spent time looking at, um, the image generation pipeline, uh, because our target model isn't like just one forward pass. It has a, um, repeated, like, uh, iter of inference. Um, which, so I wanted to try and understand how like, uh, the Jedi structures, um, how, like how they, how their iterative execution pipelines worked. Um, and so one of the more like, uh, useful insights from this phase that I got was seeing that the VLA uh, action generation path is a lot like structurally closer to a, uh, diffusion style generation, more than like an auto-regressive one. Because in the fusion, you initialize noise, then you repeatedly refine your latent and then iteratively denoise over time steps. And in the VA model, it's kind of similar where you have to initially, you know, uh, uh, initialize noisy actions, then repeatedly run the DIT action model, and then iterably refine, um, the action trajectory. Um, I thought this was pretty important because, um, it would, it kind of helped me reason about like where, um, like the problems or the key solutions for the runtime and expert issues, we're gonna, like, maybe come up later on. So reading about it was like fairly helpful. And so after building that, like mental model, I tried to, I moved into, um, tracing. Um, uh, like the actual inference structure. So, um, what I did was I traced the inference pass starting from the, uh, the libero and inference script into the uh, main model framework. I, what I did was I went from, um, uh, the model backbone, the Quen model, uh, which kind of produces, um, I found hidden states, and then those hidden inputs go, um, sorry, those hidden states become the, um, vision language

embeddings. And those embeddings are then passed into the DIT action head, which, um, runs like a, um, a denoising loop to to generate like a final action chunk. Um, and then, so from there, um, I guess what was kind of confusing to me is what I wanted to understand was, um, where the actual like runtime, like heavy like paths were. And so I traced it and I found that it was, um, in inside the predict action path. So, and inside the denoising loop, it was the model initiates noisy actions. Uh, then, um, encodes like current action trajectory. Um, builds the token sequence, runs across attention on the VLM embeddings, predicts the update, and then uh, goes through like a, uh, oiler integration. And, um, like, I spent a lot of time, um, Tracing this like pretty carefully, just to try and optimize things and note, like, which part of the system are, like, actually repeated across the different timestaps, which is fairly important for our project, like later on. And so, like, what I found is like the same DIT structure is repeating, uh, repeating across like different denoising time steps. So the dominant runtime bottleneck is not the, like, VLM backbone itself, but the actual, like, repeated execution path that we have to look for, which means, like, we're looking at more so, like, attention, normalization, uh, feed forward layers, and uh, memory movement, and then, of course, like, then repeated kernel dispatch. Um, After that, I had a look at the, uh, 88 layer norm findings. So, uh, because I identified, like, which DIT path was like the repeated, which was repeating the most, I went like deeper into it. Um, um, I looked, I read about, um, my cross attention implementation and specifically focussed on, like, um, normalization and timestep conditioning. Um, and I did this because I, I, I looked it up and I found that they were directly related to 88 uh, layer norm fusion. Um, and which was, which is something that I'm going to look at like later on the project. So, uh, the main pattern I found was, um, at the time step embedding goes through a linear layer and gets split into, um, uh, into like a scale and shift. And then, um, and then, um, hidden states, and then, and then his, his, his could, like, modulated through. Um, and I found this pattern, like both, apparently inside the Transformers, and again, like near the output production, which, which I focussed on because I wanted to understand whether the ADA layer norm was, um, actually, like, like if it was, like, really important to the runtime path, or is it just kind of like a, an extra component that didn't really affect it that much? And I guess I found that, um, it directly appears and is repeated inside the denoising transformer path, which, um, which I guess if I just leave it, it would just, uh, decompose into like a separate layer and then multiply and then add operations, which would then like overheat and then get repeated a bunch of times. So that's something that we have to, um, be careful on. Um, so, which I technically, like, yeah, it was a good look find because, um, it kind of made my proposal more like concrete. Like, it's not just the, um, like we have, we now know whether we're targeting like repeated execution paths. Um, and so, uh, next was kind of looking at like export blockers. Um, After, uh, I moved into, like, validating the export blockers from the proposal. I confirmed, like, 4 main ones being the uh, Python denoising loop. Um, the, uh, internal, um, torch, like pie torch, uh, random structure. Uh, the embedded autocast context and the batch feature ones. The biggest issue I found was in the Python DiNosing Loop. The reason I considered that the primary blocker is that exporting the full, uh, like predict action function would include like the entire itera flow on every like repeated like action. So since the same DIT structure executes repeatedly across different time steps, I found that like, if you trace the whole function, um, you risk, like, again, like unrolling multiple copies of the Transformer graph, which, so, which obviously would cost us a

lot of like runtime. So instead of trying to export the full function like immediately, I decided to kind of isolate the repeated compute and then validate whether the core transformer was exportable independently based on like the logic. And for the other ones, I just kind of wanted to move um, random initialization outside um, the graph boundary. Um, avoid like any um, precision context and move towards like just cleaner uh, interfaces. Um, and then that kind of kind of led me to, um, the next step, which is single step, um, single step. DIT export strategy. Um, the main decision I tested was like whether I should separate orchestration from like repeated compute. Instead of exporting the full denoising loop, I built a single step wrapper. Um, whose inputs are, of course, the via alum beddings, the actions that we currently have, the current state, and the timestep. Um, The rapper runs like one denoising step and then returns the predicted velocity, update every single time. The reason I did this was that I wanted the smallest stable export. Uh, I wanted to get like the smallest stable exported unit 1st because that then helped me, um, get like a much clearer, like, graph boundary, and then I, it would help me avoid, like, any Python control flow. Um, which would get, which have, which would have like affected like accuracy of results. And then it helped, like, avoid, like, internal, um, initialization, and, like, it helped create, like, a reusable, um, compute graph for later on. And then obviously, like, um, later on, it will it will make like debugging and profiling much easier as I move in to the project. Uh, and that experiment was like, I'd say, like successful. I was able to generate, uh, the inter the intermediate representation for a single step DIT wrapper. And I think the most result, uh, most important like results from this experiment was that the core transformer compute, uh, graph is exportable. And the main issue is that the orchestration or control flow around the graph, um, That, like, that's the main issue rather than the Transformer compute itself, which helps us, you know, like from later on when we focus on this project more, it helps to know that. Um, so I guess like the current direction that we're taking is, um, like keeping the schedule, like external, and then compiling the repeated, like, DIT step as a reusable for, like, the open veno graph. So we're avoiding like dealing with repeated code all the time. Um, Next, um, I I did some, uh, intermediate representation, like validation, and, like, runtime execution. I didn't want to stop. I just, like, generating, like, bin files. Um, because those are, I think, uh, as I read, like relatively misleading sometimes. Um, so the next thing I did was to uh, validate the, um, or make sure that the, the, the, uh, intermediate representation could actually like load, compile, and then like execute through the open vino runtime. I started inspecting the operator structure because I wanted to, um, understand how patterns like, um, 88 layer norm and attention are represented after conversion. The reason, the reason I think this matters is that understanding whether operators stay fused or become like heavily decomposed, will, uh, I think, be probably better, uh, especially when we're looking at profiling optimization. Um, The validation itself was pretty successful. Uh, like the intermediate loaded, loaded, and then compiled on my CPU, and then executed, um, with the expected like tensor shapes that I, that, like I predicted to get. And then after that, um, I started setting up like, uh, like validation. Um, to confirm that, um, the open veno runtime, I'll put like matches what the uh, pie torch wrapper output got before building like more experiments around it to make sure that we had like a good baseline and that we weren't like, like starting off with a batter or incorrect base pretty much. Um, Yeah, and then after that was, um, I moved more so like architecturally, uh, like, okay, like now we know what the problem is. Um, can we

construct the full denoising behaviour externally around the uh, compiled graph? So I, uh, built like an external loop that repeatedly caused a compiled, like, open veno single step model. And the reason I wanted to test this specifically is that it directly validates the architecture direction from the expert blocker analysis. Um, using external orchestration and a compiled, um, like reusable DIT step, again, like instead of just going through multiple uh, loops of code again and again. And, uh, obviously, like right now it's just still like a, like a prototype, like a, uh, I can't confirm if it, like, works 100% yet. Like I still need to do some more pre-processing and pie torch. Um, but I think the important thing is that, like, the idea of the repeated transformer execution is now like handled through open, you know, one time execution, which is pretty good. Uh, and it gives us like a better structure because like our scheduler becomes like more, uh, lightweight and and, um, especially in like the, in terms of like orchestration, and then the heavy, like reusable stuff becomes more like a compiled graph. Um, which, again, will help later on for profiling an optimization. Then after that, I moved to kind of like a benchmarking. Um, uh, so I know you guys asked me to like look at baseline, so I, after that, I started building like the infrastructure for that, and I guess the idea right now is, um, is not to acclaim final performance numbers because I'm still like running them locally. Um, and not on like Intel hardware. I actually did try to set up the cloud through the link that you guys sent me, but I'll show you in a minute, but it wouldn't let me, like, set it up. Um, well, I, I, I got, like I set up my account, but it told me, um, that I had to have like an intel or employee account. So, but I'll show you guys that in a sec. But just to say, like, uh, these numbers are from, like, my laptop and not like Intel hardware. Um, so the goal was instead to kind of try and establish a, uh, something that's a benchmark that's like reproducible, and that mimics like good warm-up behaviour and then that can be repeated, and then that can also compare like latency between the two. So I benchmarked the pie torch, uh, single step wrapper, the open vino single stuff graph, and then the repeated, like, external roop. And I want to do this like now. Obviously, it's like, um, once that, like, I, hopefully I can get like cloud access or GPU execution, then I can just reuse the same scripts and I wouldn't have to like redo this, uh, later on. So once I hopefully have that, I can, uh, maybe compare, like, runtime behaviour across, uh, CPU and GPUs or pie torture, open vino. Um, yeah. And so, Um, that kind of actually, no, I, um, in terms of what this proves, um, so overall, I think that the work so far shows that there's a few important important assumptions from the proposal that we have to stick to. First being that the VLA architecture, uh, actually does behave like a diffusion style system. Sorry, diffusing style, like denoising system, which will help us in terms of the structure of our solution later on. The 2nd thing is that, um, that like the dominancy of the runtime heavy path comes from the repeated DIT execution and not from the VLM wrapper itself. The 3rd thing is that the ADA layer norm is directly inside, um, the denoising path. And so it's important that we target that too for like an, like a, like a solid, like optimization target. And then the 4th thing is that we can export our DIT graphs to our open vino intermediate representation. And the last thing is, of course, that the external scheduler plus the compiled, like, um, reusable DIT steps. Um, like the architect, the architecture appears as like the 1st, like that should be like our next step. So, um, That being said, of course, this is this is what everything that we proved, um, that I talked about. Uh, yeah, so again, like, uh, we're good. The model matches the proposal. The runtime, we now we know where the runtime live. We know where the problem is. We know what we have to target to

optimize. We know, um, we know that we can export it and we know that our strategy is like still, like, we haven't had to make a shift in the proposal or the initial plan that I made. I guess just yet. Um, in terms of, um, next steps, uh, my next steps are, uh, you know, finish finishing like any, like, any more like, uh, validation. Um, cleaning up, external, like, uh, denoising loops. Um, obviously, looking at the cloud axis problem, and then bench the benchmarking on um, Intel hardware to see if we, you know, get the predicted results. And then inspecting the exported, like, intermediate representation more carefully once I can do that. And then we can begin like profiling. Uh, based on, like, like, based on the experiments, I think that's kind of a solid, um, approach, like, the reason I'm focussing on operator expansion before optimization is just so I can understand how the runtime, like, represents the, uh, the DIT path. before deciding where, um, optimization would actually work. And the next major step after that would become like profiling. And at that point, I think my, my, like, idea, um, like, yeah, my idea is that the repeated, like, DIT execution path, um, will help, like, like, will dominate, like runtime. So I think that will help a lot with optimization. But I want to like validate that with actual like profiling traces before making like any solid um, optimization decisions. Yeah, that's kind of what I've been doing for the past, um, 2 weeks. Other than that, of course, I tried to set up Cloud. I've just been looking into open vino more, looking into VLA models and we're trying to brush up on any knowledge that I missed out on. And, uh, obviously, I also have that checklist that you guys asked me for. So, um, I'll actually, I can just show you that now. Yeah, so, um ... Yeah, so I have it in like, this is kind of just like a general page. So all of the tasks that I predict having, um, having had to, like that I will have to do are here. This is kind of like what I worked on and this the past 2 weeks. These are, like, things that I would need to do once I have, um, you know, like, access to Intel hardware. Um, and then these, I left them here as pending just bec- instead of like in progress because I, uh, I'm expecting to maybe have to change strategies or add something depending on some of my output. So I'm leaving them here as now. And then in terms of next steps, I guess, like I discussed, I should. Um, um, I will kind of be tackling like these next 8 things. So again, like finishing any validation. Um, finalizing, uh, any, like, implementation, getting on, uh, intel hardware and then benchmarking and then, you know, going into, um, profiling pretty much. And I'll make sure I can shit, I'll share with this with you guys. I can also try and move it to boo-boo sheets, so, or I can just send you, like, if ever I make a change, uh, I can probably, like, yeah, I can probably share it through one drive, so if we make any changes, uh, we can go over them. Um, but yeah. And yeah, that's pretty much what I've been working on. Um, what do you guys think? So, I'm thinking ... Sorry? Okay. I'm sorry, it's been shiner, it is. Oh, yeah. Yeah, uh, It's, um, can you best share, um, fish has, uh, large, um, The presentation? Uh, no, no, the Intel Powder. Yeah, so I'll show you guys the actual link. Uh, it just basically told me to sign up and then, um, And so, like, that worked just fine. And then, um, And then, but once I got into it, it just wouldn't let me, oh, sorry. It wouldn't let me actually, um, Oh, would it go? Sorry, one sec. Okay, I'll open it open again. Think I may want to close it by accident. Um. Okay, so here I'm, like, signed in. I can sign in again. But, um, So is, count on a little bit, but ... Yeah, it said, it said, um, okay, I don't know why it's taking so long, but basically, when I signed in, it just kept telling me that, um, I have to be an infel employee to actually, um, I don't know why it's taking so long to to load, but, Um, yeah, pretty much it just said, like, it was exactly the same screen, and

it just said, like, uh, you must sign in with, like, an employee account or anything or something like that. So I couldn't, although it did it didn't make, like, let me make an account using my personal email. I just, I can't see like any of the dashboard. I tried to go through like another link or a different website to go to try, but it all like just ended up taking me to the same place. Not like signing out, but ... Maybe try and order it again. I can't wait and then to find it. So I'm on a ways for you to get the results. From involved. Sorry? Yeah. Oh, um ... Think, um, it comes out, uh, Um ... Complayeros, when we ... Wait, uh, we cannot, um, provide you directly the, the device, fast, so, uh, let me. Let me try to figure out is, is there any other news from you? Okay, so, uh, sorry, just because the audio is not like the best. You said you can't give me the, um, you can't give me an Intel account, so you'll try, you'll try and figure it out, and then get back to me. I think, uh, I will, uh, try to find other, uh, other ways. to access, um ... Okay. Is healthy. Oh, yeah. Do you have? Do you have any, like, suggested ways or should I just start looking into it? Oh, my dear. I mean, all about that, I mean ... So, I don't know, I think you, uh, you can. Maybe, and try the model conversion. During this Spirit. Uh, you, you just converted the data step. So, uh, you, you can. Next step, you can try to convert other components. Just like LM for VIT. Convert other components into what? Sorry? Into our domain though? Okay. Well, do you have, like, which ones are you mentioning, like, specifically? Or if, or do you want me to just mean like explore more? So, I think uh, I'll finally go is to, then all the, on that, like to, you know, ask. So you, you need to, uh, ex, to convert on all the package models to the, um, Not just DIT. Yeah, 2 books. Okay. Okay, I can definitely, um, I can definitely do that. Um, so what it, in terms of like my, um, progress like this week, do you guys think I made enough progress or would you expect me to make like, like more tracking or, I guess, not more like a, like make better progress within the time frame that I had? Um, I think it's, uh, less enough for you to. Yes, no. Sorry, can you say that again? It cut out a little bit. Um, when we think the progress is good enough, and I want to, can I ask some question about the slides? You share for? Sure. I have some technical questions about it. Sure. Yeah, um, so, uh, 1st time, uh, you decide the model structure, right? And I, H. 85, 84. Uh, the model sector here? Yeah. Uh, have you tipped out the, uh, tip? Tip up the, like, monster structure, like, uh, what's the config of the model? Uh. For example, um, this very model is composed of the VIT and the mm mm model and then the IT, the action model. Uh, so how about the, like the hidden size, intermediate size, uh, of the transformer and the input shape, output shape, uh, something like this, uh, Have you summarized those data? Um, I think I did. I can look through my notes very quickly. Um, I should be able to find it. Yeah, why I ask it, it course, um, those data where, uh, without the, uh, uh, uh, Those, those comic, all those data, um, waiters, how large the workload is. That is, um, uh, just directly related to the final performance. That is, um, if, uh, so before we hedgehog the baseline almost, we need to uh, fix the perfect data or the info shape or the uh, scenario, or like some target cases. Okay. Okay, yeah. Yeah, I think I have it somewhere here. Uh, I look through my notes. I do, I definitely feel like I I, I looked into it. Maybe I didn't document it, um, as well as I should have, but okay. What I have here is, um, so, like, in terms of core dimensions, I just have, like, like hidden dimensions for, uh, the Quinn model. Like, I think I got like a 2048. Uh, got a GIT input dimensions and inner dimensions. 1st one being uh 1536 and then 1024. Um, I have uh, DIT depth, uh, which looked as uh, like 16 layers, and then, like for the attention heads, there was like 32 heads per layer, and then in terms of inference

time steps, it was, um, 4 steps. Um, which is, I guess, like a default for flow matching. And then, um, in terms of tensor shaped, I, I, I got like, um, Um, Sorry, I got like for the VLM embeddings. I got like one, 512, and then 2048. So, like, uh, I guess like batch, then tokens, and then, uh, features. So five, one batch, 512 tokens, and then 2048 features, uh, for, yeah. So, we move on to, like, uh, we have a target case like we, uh, we want to, uh, free camera or 4 cameras, input. of the imaging of the imaging put, and then about like, uh, Then, uh, through this model, we do the 1st we do the vision coding and the, uh, we are at, uh, VIM auto and then go to the follow-ups, the IT action head, and here is, uh, I see here, you, you, you see, the bottom neck hypothesis, is, uh, is for more about the repeated GIT, as, cushion. So, um, um, that's, uh, uh, when we list all the conflicts of the different component model, like VIT uh, launch model and the action model, and uh, uh, We, we see, they have different, um, Uh, uh, uh, Like, uh, VIT is about to, like, uh, uh, 100 or like 3000000 parameters, and the way I don't like about to be, or even more or even less or less, less or more, like the such uh, parameters, and DIT just about like several 100s of, there were 10s of 1000000 parameters. So why you think DIT is the bottleneck rather than white language model? This language model may be much bigger than DIT, even DIT West follows. Um. Well, based on what I found, um, it's, I guess like, uh, I, I figured it was probably like, uh, the balling, just because I, I looked at the amounts of like repeated like iterative, um, like solutions that I, I mean, code that I found. And so I figured that that must be, um, a lot of, like, where the kind of, um, that, uh, that the compute was coming from on that end just because, um, just because, Because it's just repeating, so it, it, it's more like a, like, it's almost like, like, everything else is essential because it runs through once, and my goal wasn't like to optimize, I guess, the model was like the performance. So, like, like, I guess in, like, in terms of, like, robot inference, um, like, I looked at, like, like, the iteration multiplier, which, um, because it, like, for the VLM backbone, it runs like once per observation. And, um, encodes like the images and the text into like, you know, embeddings. Whereas like the DIT action head runs like that like end times. Um, well, depending on like the de noising steps. And and so, like, even though the model, like you said, is like a lot larger, like the cost is paid once, right? In terms of compute, whereas like the DIT costs, because it runs like an X amount of times. So you just multiply it like by X always. So, yeah, so I guess, I guess my idea was like, um, if a single step took like, you know, like X amount of time, then it would be, for the VLM, it would just take X, where, sorry, not X, but like, uh, it would, basically, yeah, like I said, it would just, it would, it would just, even if it might be higher, still a fixed cost, whereas, like, eventually because the DIT runs more, it could eventually cost more or count more, like, I guess, for more percentage than it should be if that makes sense. Um, yeah, but I, uh, what I want to highlight that, uh, the loops or number of the steps, and not the only indicator to uh, measure the workload or the bottom neck. In my experience, like uh, I'm working on uh, on the optimization of uh, high 05 or group, this model. Um, in that model, uh, we see that, uh, the in 1st time of Royal M model, It's larger than, uh, much larger than one step of the IT. So when we see the, when we, Compel the, uh, where, where I am in 1st time, uh, with the total action model inference time. Maybe, uh, About half, uh, 55 like this. So, Mm. Um, I just want to, uh, tell you that. Yeah, team may not the only auto neck or not the most important bottleneck in this model. Uh, you may need to look, look into the best time, or uh, when you have a machine to do bedmark, or you can have overview of the complete data, uh, and to see how, uh, how those configure data will impact the

final problems. Okay, so you're saying, like, okay, like, I should shift my focus, like, rather than, um, considering, like, the DIT action that is one of the main, uh, kind of bottlenecks is kind of shifted to the side and then focus more on the VLM itself. Uh, not shift the focus, but you can, uh, not only the GIT. Okay, so look into them both. And you think that the, like, I guess, profiling and optimizing the VLM will help more than the DIT, or are you saying it's like just, like, as well? Just so I understand. Uh, so, um, I know you just, uh, hypothesis, uh, right now at the current status, because you do not have such benchmark A-Tag. So it's okay. And also I think it makes sense. It makes sense to think, uh, GIT, maybe the bottom access, it be around several times. Um, and, um, for we are a model action, It's also very unique and, um, A unique part to differentiate with other, like, large model or other scenarios. So I think it does make sense and but I want you to have a overview of the BLA model rather than owning the GIT component. So yeah. Okay. Okay. And so, and the, and the, another question is about the, um, Are you moved to age 7? Uh, sure. This one? Yes, um ... What did you mean, slide seven? Yeah, yeah, yeah, yes, I said. Yeah, this page. You see, it's also okay. Um, so you, uh, for the dad part, you move the WhatsApp, uh, like you, you move the loop, external. To the auto. So you will note for a remodel of GIT, when, when the impress, right? Right. And so, um, Have you considered that the load overhead or the? Like the, some bubbles between each step. Some what? Uh, no. Sorry? But, I think, um, Of course, you, um, convert one step, GIT to one OV model, right? So when we run 4 steps, then you need to load 4 models and then write one by one. Oh, I think, if he is in the use, one, one model. I raised one model and the compiler ones. But do you run the model 4 times? Uh, yeah. Uh ... Sorry, I think I got a bit confused. So you're asking, like, how many times I, uh, uh, ran the compile? Uh, I think you, uh, you covered one single step into a window IR, so you, uh, Yeah, well, I guess my idea was to kind of, um ... Loop one, this one model. What times? I should loop. I should I should loop it multiple times? Uh, can you ... Later on, when you run the end to end inference of this VOA model, you will. Of the one step model. For what times, right? Right. So, in the end, he overhead between you one, 4 times. On the IR modern, uh, comparing to one model, but with almost internal. Uh, okay, so, you want me to, like, move the logic into the IR and then, and then, um, Uh, sorry, I think I, I'm a little confused. Oh, wait, uh, we think you, you can have a try to, uh, look forward a model that, uh, Includes the, the followers in internal. And you can compare them. For benchmarking and you, you can see some different. Okay. Okay, okay, okay. Okay, I got it. And, uh, I, I'd love to, I will not get the reason why you move the one step. Um, I think, um, we mentioned about the runtime. Uh, well, I guess my idea was because it's, um, like, like on a, um, I guess, on a GPU moving back to the CPU, and then back to the GPU would, like, create, like, a massive bubble. So I guess, like, my idea was validating that the single step works and then using that, like, um, like open Veno loop as an operator to, I guess, keep it. So, like, I guess, like, I guess for now, um, um, I want to keep it there until, like, the execution was entirely on the hardware. Uh, so I guess that's why I moved it to, moved it to single step, but I guess I can, I can try moving it back. Um, and just, I guess, since I'm gonna be looking into other solutions, just do that then, is, is that what it is? Sorry, which which operation will cost the model for back to CPU? Uh, I, it would be the compiled single step, no? I mean, uh, if we compile the 4 steps to one OB model IR. Right. Uh, if you, if we do like compile 4 steps in one open window model, I art, so, uh, it will run one step and then fall back to CPU and then again to attitude and so for act like that. Oh, okay. Yeah, I can set it up

like that. So, uh, you said, uh, starts with CPU and falls back to GPU? No, no, no, I mean, uh, uh, I want to know the motivation that you do. Like compile the single step DIT rather than, The whole DIT. Yeah, component, with all steps. Well, I guess because, um, um, like looking into it, um, I guess I thought, okay, like this is like repeated, uh, like this is repeated, so can I, like, kind of, uh, compile it into a single step rather than multi-step and then get the same, um, results rather than, you know, like, um, going through all 4 steps. Can I just do it in one step, um, while compiled, and then get the same result, and then what that do, because since it is like reused, like reused or like reiterated multiple times. So can I do that and then get the same, and then would that then help? Um, in terms of optimization? Because that was what I was looking at. So the benefit of that is to save the memory or like make the OV model IR info smaller? Yeah, it was to kind of make it smaller, save save memory. So then later on, we could, like, it would help with, um, like better profiling and then, um, I guess optimization, like, uh, later on, it was, I guess it was just kind of like a, an experiment to see if that would work. Well, do you think it was, like, the a wrong approach? I think it, um, it's okay for, uh, for profiling prepares or like the memory peppers, it's okay. But we see in group model. When we feel the DIT steps, the purpose will be better. So from the performance. So maybe, um, Single step, DIT, um, it's not that competitive. Okay. Maybe you can try not, uh, when you, when you have that. Yeah, we, uh, to, to, to recommend you to try the different approach. To, to see that different, um, because it, um, it depends. All different cases, um, where you can, um, we can hardly say, um, one approach is on, probably the best. So. We think you can, you can try. Yes, you can try both. Okay. I'll try both, and I'll, um, I'll try both fusing and, I guess, a single step, and then, um, yeah, I guess. Yeah, okay. I'll try both and then I'll, um, benchmark them and then I get, so I guess like the main thing I have to do is make sure I have that, um, hardware. setup. Is there like, do you know if a way of, like, that I, that typically people would go about other than using the cloud or, um, or I guess, like, one other strategy is like, is there like, uh, like, in similar to how I could do it with, like, um, Nvidia GPUs? like I could rent a pod, like does that work too? Or not necessarily. It must be Intels, IP. No, no, no. I meant, like, I meant, like, uh, maybe old, old Russian, like, um, I meant, like, do you know, like, a website that rents out, like, um, Intel IGPUs, similar to what, like, how, like, I can rent out any video on? I can look, I know like of like, okay, I'll show you the one that I typically use for like personal projects. Um, Oh. Uh, it's called. I think it's really hard for us to rent, uh, Intel device. Okay, so which, like, how should I, um, Like how, how should I approach getting my hands on them? No, and I think. But, um, maybe ... Oh, there's much easier. Um. It's think. Well, uh, let me think about this of black. And I will give give you a feedback. Okay. I can, I'll also look, look into, um, other ways to, um, I could also, um, I guess I could also reach out to the, uh, like, I guess, like, lead person, and, um, for, like, the Google Summer of code, like, open open Veno, uh, like, team, I can see what, like, other, other, the other, like, students are doing to access, um, IGPUs? If, if they're working with that too, I could, I could reach out. Okay, okay. Okay. Um, and uh, you can, uh, you can still, uh, came to the little conversion because the, this task is not part of a specific, Okay, you can investigate the pythology model. Uh, architecture. Yeah, I'll definitely start, like, setting up, like, some of the code and the structures and reading up on that. So that way, like, by the time I eventually get, like, access it, I just have to like run it and like get my results and and then compare and then retry. Have you created Apple for this

project? So can you share the link? A repo? Yeah. I know I have a repo. I have to check if I pushed everything that I did. Um, But I'll share, I'll share the link with you guys anyways. Uh, there you go. Yeah, I might have to push. I have to push something, so I'll do that like right after. Uh, but I'll share it with you guys anyways. Here's the depot link, and then I'll also put the, um, I'll put my, um, my slides and the tracker. Are you guys good with, like, Excel? Would you prefer like Google Sheets? I can also just, like, basically copy paste it into sheets if that, if that's easier in terms of updating. Which one do you prefer? Anyway, okay. I don't know, like, I think I'm more used to using, like, Google stuff because, like, I don't know. I've never used, um, like I know like you can use like, uh, um, uh, one drive for like cloud stuff, but I've, my experience is that it's easier to do with the Google Sheet, so I could genuinely just like copy paste all the stuff and then send you the link like right after this call. If, and just send, and then, um, that way it's easier for, like, you can just go onto the link, and then if I ever change anything, because I clearly have to update a few things in terms of, like, my targets and stuff like that, so I can make those updates and then share the link like today. Yeah, I think, well, okay, we the, we don't, we should, we can share some link, and then we can check if we have the access. Okay, sounds good. And for now, I'll send you guys, um, well, I'll also send the slides through. I think I can't do it in the call chat, so I'll do it like right after two. Um, yeah. Uh, and the one working I want to ensure that uh, maybe you can um, validate the option level, practice of the output when you cover the open model. Comparing to petage results. Valid the, you said the option level? The, The function. Oh, the. validation. Okay, so validate the function level against PyTorch? Yeah. Okay, but I'll do … I think the, the, yeah, like compare the output, tensor of the, uh, OV for the, with the head touch. Okay. And I think the standard is that the NSA is still be less than. It should be less than, um, by how much? Or do you want to just see that, like, you just, oh, it should be less. Wait, okay. don't know how to say it in English. Uh … I think, uh, less than, there are a point. One% um. Yeah, yeah. 0.1%. Should be 0.one point 0.one% faster? Uh, it should be less than 0 point. Deposit one. Oh, okay. Oh, oh, like, oh, less than, okay. Okay. Um. Okay, cool. Oh, note that down. Uh, Okay. That's not, is that like a, is that like really fast or, um, because isn't 01%, like 0.one%? Like, isn't that like not that much difference? So I guess like, I guess like, that's what we're kind of trying to aim to improve. Okay. Okay, cool. Um, do you guys have any other kind of feedback for me on like what, how I did or, um, what I should be focussing on, maybe in terms of like my approach to doing everything? Mm hmm. Um There were some, maybe? Okay Yeah, I, I think, um, it might be. Maybe I'm anyone, it's at her. Sorry? Um, yeah. And also the standard, I mean, for an AE. When you test the, when you do the vendation of the output. Oh, oh, oh, yeah. Uh, MAE and MSE. Um, I'll be completely honest. I kind of forgot what they both stand for. I know what they are, but I forgot what they saw how it looked them up to. I know, 1st one must be like mean standard error, I think. Yeah, yeah, and then the 2nd one, I can't remember, but I'll look it up. Excellent. Absolute. Oh yeah, absolutely. Okay. Okay, so the standard error, less than one, uh, 01, 0 should be one, 0.1% faster, and then, um, the mean absolute error should be one times 10 to a -3 faster. So, yeah, thank you. Because, um, you can really focus on the Modo level. Yeah, I'll focus on, uh, like. Oh, that's for sure. um, the, the, the, the, um, Don't worry, level. authorization. So, um, I think, um, You can, You can focus on both view and and uh, oh, actually expert. I can, sorry, one more time. Uh, both where I am and the DIG component, and you can also try to combine it to it.

Into a full model. So And you can. To the end to end. Okay, model covered. Yeah, you cover the, the, the whole year into a, A single over middle air. I think you can also try. Uh, so sorry, just, I can, I could also what? Um, covers the last song. We are a model, into a single. Over middle R. Right? Was there not something after that? Yes, sorry. I asked like, was there something, um, that you said, like right after, like, because, you just said, like, convert, like, end to end, the VLA model into a single open vino app, right? And then, and then you said like something else after that or no? Mm. Mm-hmm. Um, I just want, I just want to say, uh, the performance and to anime, like a different. Oh, okay. The whole, uh, open middle IR and the, The several hours. Combination. Yeah, they they need really different. Okay. I'll look into that too. Um, yeah. Um. Yeah, I guess, yeah, yeah, I know what I'm working on. I guess if I come up with anything, uh, any questions I can, um, uh, ask you, I can, I can just message you guys, right? Yeah, yeah. Okay. Yeah, but overall, in terms of like progress, uh, at the current pace, I'm doing okay. Yes, I think so. Do it with … Okay, um, also, I guess, like, I forgot to, I should have asked, like, at the beginning, but, um, like, uh, do you guys feel like I speak too fast or, like, I, like, I don't articulate as well? Uh, I know since it's probably not your 1st language, it's a little harder. It might be a little harder for you to, like, in terms of me presenting, was I maybe talking too fast, because I can always, like, try and slow down or pay attention, because I know it's not your 1st language, so. Um, I think it's okay. Okay, if I'm ever like not explaining what, please don't hesitate. I know it's a little difficult. I'm, my English, English is like my, uh, a 2nd language too, so I know I can speak a little funny. Mm Uh, yeah. So, it's fun. Eva, Chelsea, Conroy. Sorry? Let's give a short summary of this meeting by the progress and the ARs for both you and this. Uh, okay. Yeah. Uh, oh, yeah, so, oh, are you giving the summary or should I give the summary? A, I, I, I can give a sandwich. Um, by smoking, uh, you, you have shared us, uh, progress on, um, On your, uh, investigation on the, um, temperature and the, the, And so I'm, uh, experiment, uh, um, the, um, emotion, and. Okay, I know, uh, and while AR for me is to, uh, to figure out if there, uh, any other ways for you to access the Intel device? Uh, and then, and I will give me a feedback, you know. Okay. We have … We have some comments for your, for best, and we, we're not sure. Help you. Better on, on your, for the progress, so, um, so, uh, you can, you have tried, uh, you know, try, uh, dress up, we, we talk about the, um, the, What? I don't know. what I wanted to say. You guys could try, um, I don't know, like Google Translate, I guess if that could help. Oh, um, I forgot to, to ask you, um, do you have any questions or any help from us? Um, well, so far, I think I'm, uh, I'm good, like, uh, I think, like, um, I had a few questions, but, um, like, it was more like about, uh, my approach to things, like, uh, uh, yeah, cool. We answered a lot of them, like, like, I was gonna ask if there's any, um, other bottleneck category like you want me to look into. So we talked about like, uh, looking at the actual like uh, VLM model as a bottleneck too. So I'll look into that. Uh, I was gonna ask uh, about, like, uh, in terms of, like, um, like your experience, um, is there like a specific, like, boundary that you, like, in for runtime that, uh, that these pipelines, like, have or would you generally prefer for me to keep, like, orchestration and, uh, like, um, like, like, keep, like, external for debugging or like, push more into, like, pre-processing for like the compiled graph. Like in your experience, like which is better. Um, what do you mean boundaries? Of the runt? Like, Uh, like, uh, but basically just asking like, like, like, basically which, which one is better? Like, uh, keeping orchestration external or, uh, like, I guess like in practice, like, is there one

that's better? Like, the, because I, I, I think I lean towards keeping it separate from what I found. Uh, or should I, do you think it should, it's better for me to push more into like pre-processing, pre-processing into the compile graph? Or is that just kind of like a variable depending on, um, um, what the situation? Um, I think in our, um, expanding, um, experience. We refer to. To compel the model. Okay. I'll just keep that in mind. Oh, sorry, go on. We can go. Oh, that's a different. Oh, okay. Oh, sorry, sorry. Oh, sorry, I'm sorry. Yeah, so I guess I'll just keep that in mind. Um, in terms of like which one I lean towards and then, but I'll still like try and analyze it given the situation. Um, but yeah. And then other than that, I was also, I was gonna ask if the, uh, like compiled single step architecture is the, like, uh, is the right thing, but I guess you you said like I should try, uh, I guess generally like fusing it rather than the single step. So that's good. Um, uh, let me see what else I had, uh, I had, um, You know, we, we, we prefer, um, to fuse the component as much as possible in the architecture. So. Um Is there, like, a specific, like, reason or, uh, like, is, is it just based on an output? Um There is a reason, um, that, that, the aluminum, can do some, uh, graph authorization, infuse the kernels. To get better performance. And uh, and some other, uh, other banishes like uh, has a memory layout. Uh, If you feel the model as much as possible, you can, okay, uh, guess, uh, the, Um, you can get some manages from the, uh, The compiler and the telephone influence framework. Uh, they have some stress strategies to, to fill the, the, the kernels. And to, to reorder the layouts, the, the cancer layouts to get that better. So, uh, well, rather than you sleep, the model into that work on it. We, we think you. You can, you can, and personally try the, the, Feels version. Right. And, uh, Yeah, and this is based on our experience. All other models. Okay, cool. I'll definitely try that. Um, Other than that, uh, I had, um, Um, I guess. Yeah, if you, if you found there, there are some boundaries, uh, that can, cannot be, uh, directly, come, uh, converted to a, you can try to uh, modify the petage, the petage code to bypass it. I think. Wait, can you say that again? Uh, you can, you can try to modify the path touch code. To bypass the boundaries that can. And not be erupted, converted, for coming us. Right, okay. I say you mentioned the batch feature. The class batch batch fisher in your slice. It, uh, it may cannot be, uh, a virtual, you know. Oh, you can modify it. Okay. The one more thing I want to, uh, mention that, uh, maybe you can, Um, Do something about your flight. So can you, have you started the roof of my anthology about to, Mm, measure the, Theoretical. The what, sorry? model. Roofline. For one performance. Sorry? Um, blue flame, performance, um, I think you can, you can start with, uh, try to compute the, The, uh, The, uh, The Rotico, um, computation, Commitation, uh, tea flops, or some of the subject, in this model. I could start. What is the upper, the upper limit of the performance? Then you can know how large the headroom, uh, to the upper limit. To optimize the model. So, you want me to find, like, the roofline performance to find the upper limit of the model performance so I can try try to get like a more concrete goal to Target. Is that what you meant? Yeah, and uh, how much, and how large the happy you can. And You, you have to, to optimize the model. Oh, like, like, you mean like, like what percentage higher I can go? Okay. I remember you said we were aiming initially for like 30 was it like 2030 or was it 3040? I think it was 3040, right? And then ... I think 2013 or 2014 maybe ... 20 to 30. Yeah. Okay. So you can't. Sorry? Yeah, it can ... Okay. And then you have the hardware, you can compel the baseline of best window fly model and to see how much the room you can. There is all your optimization. Okay, cool. Uh, yeah, okay, I guess, yeah, I can save, uh, some of my other questions. Um, uh, in, in regards to like, uh, when I get, when I

finally, I guess, start like more profiling stuff, so I think it would be better if I just save them for that. But for now, I think I have like a solid, uh, few things to approach for the next, uh, 2 weeks. Okay, a minute. So, are you saying? Sorry? Oh, no, so I think, uh, it's all, uh, for today's wedding, and, uh, I think, uh, I don't know, we can organize, call, I don't know, high board, airs, and, uh, um, let's name it, uh, 2 weeks later. Okay. Sounds good. Yeah. Cool. Thank you. Buckle, we'll send the meeting notes better, like to highlight the AR's book, and also the comments for you. In case you, Captain, capture the comments. Okay. Okay, sounds good. And I'll send through my presentation and then today I'll update the checklist and I'll send it through. Okay. Both. Cool. Yeah, start with progress, and, uh, espoil. Thank you guys. Yeah. And let's eat. 2 weeks later. Yeah. Okay. Yeah. Sounds good. Thank you guys. Have a good 2 weeks. Have a lovely day. Bye. Yeah, so I'm also making sure I'm checking the, uh, mean standard error. Against, I have to validate the, uh, function and check the, that the mean standard error is less than 0.one% and then the, the mean absolute error is less than 0, um, one times times E to the -3. Um, I have to also work on model conversion, make sure it's, uh, Um, Yeah, make sure and check like roofline performance, make sure it's, you know, 20 to 30%. Um, and then work on like, fusing, fusing the kernels and fusing, rather than just doing a single DIT step, doing a fused one. Um, Pre-processing and …